

A word identity does not capture word meaning

Lexical meaning includes senses, relations, and connotations.

Form and sense

LEMMA

mouse

WORDFORMS

mouse · mice

SENSES

rodent

cursor-control device

A word identity does not capture word meaning

Lexical meaning includes senses, relations, and connotations.

Form and sense

LEMMA

mouse

WORDFORMS

mouse · mice

SENSES

rodent
cursor-control device

Meaning relations

couch $\approx$ sofa

synonymy

cat $\sim$ dog

similarity

coffee $\leftrightarrow$ cup

relatedness

A word identity does not capture word meaning

Lexical meaning includes senses, relations, and connotations.

Form and sense

LEMMA

mouse

WORDFORMS

mouse · mice

SENSES

rodent
cursor-control device

Meaning relations

couch $\approx$ sofa
synonymy

cat $\sim$ dog
similarity

coffee $\leftrightarrow$ cup
relatedness

Connotation

innocent / naive

Similar reference

Different evaluation

A word identity does not capture word meaning

Lexical meaning includes senses, relations, and connotations.

Form and sense

LEMMA

mouse

WORDFORMS

mouse · mice

SENSES

rodent
cursor-control device

Meaning relations

couch $\approx$ sofa
synonymy

cat $\sim$ dog
similarity

coffee $\leftrightarrow$ cup
relatedness

Connotation

innocent / naive

Similar reference

Different evaluation

How can a computational representation capture these patterns without listing them by hand?

Context provides evidence about word meaning

Words that occur in similar contexts tend to have similar meanings.

Unknown word in context

ongchoi is delicious sautéed with garlic

ongchoi is superb over rice

ongchoi leaves with salty sauces

Context provides evidence about word meaning

Words that occur in similar contexts tend to have similar meanings.

Unknown word in context

ongchoi is delicious sautéed with garlic

ongchoi is superb over rice

ongchoi leaves with salty sauces

shared contexts



Known words in similar contexts

spinach sautéed with garlic over rice

chard stems and leaves are delicious

collard greens ... salty leafy greens

Context provides evidence about word meaning

Words that occur in similar contexts tend to have similar meanings.

Unknown word in context

ongchoi is delicious sautéed with garlic

ongchoi is superb over rice

ongchoi leaves with salty sauces

shared contexts



Known words in similar contexts

spinach sautéed with garlic over rice

chard stems and leaves are delicious

collard greens ... salty leafy greens

garlic · rice · leaves · delicious · salty

ongchoi is probably a leafy green

Context provides evidence about word meaning

Words that occur in similar contexts tend to have similar meanings.

Unknown word in context

ongchoi is delicious sautéed with garlic

ongchoi is superb over rice

ongchoi leaves with salty sauces

shared contexts



Known words in similar contexts

spinach sautéed with garlic over rice

chard stems and leaves are delicious

collard greens ... salty leafy greens

garlic · rice · leaves · delicious · salty

ongchoi is probably a leafy green

How can we turn these recurring contexts into a numerical representation?

A context window defines what gets counted

Here, context means four words to the left and four words to the right.

Every target occurrence contributes one count for each word inside its window.

is traditionally followed by **cherry** **pie**, **a** traditional dessert

often mixed, such as **strawberry** rhubarb **pie**. Apple **pie**

computer peripherals and personal **digital** assistants. These devices usually

a computer. This includes **information** available on the internet

Selected context dimensions: **a** **computer** **pie**

J&M §5.3

Before the next slide: count **a**, **computer**, and **pie** around each target word.

Co-occurrence counts become word vectors

Each row records one target word's pattern across the same context dimensions.

rows: target words

columns: context words

target word	a	computer	pie
<i>cherry</i>	1	0	1
<i>strawberry</i>	0	0	2
<i>digital</i>	0	1	0
<i>information</i>	1	1	0

A row is a vector

$$\text{cherry} = [1, 0, 1]$$

The order preserves the dimensions:
[a, computer, pie]

cherry + strawberry share the pie dimension.

digital + information share computer.

Real vectors have $|V|$ dimensions
and most entries are zero:
sparse.

J&M §5.3

How can we measure whether two word vectors have similar patterns?

Cosine compares word vectors by direction

The same geometric measure now compares patterns of word contexts.

Recall from document vectors

$$\text{COS}(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \|\mathbf{w}\|}$$

Similarity is about direction

same direction $\text{COS } \theta = 1$

right angle $\text{COS } \theta = 0$

Normalizing vector length prevents overall frequency from dominating.

Apply the same rule to word vectors

Dimensions: [pie, data, computer]

word	pie	data	computer
<i>cherry</i>	442	8	2
<i>digital</i>	5	1683	1670
<i>information</i>	5	3982	3325

$$\text{cos}(\text{cherry}, \text{information}) = 0.018$$

$$\text{cos}(\text{digital}, \text{information}) = 0.996$$

information is much closer to digital than to cherry.

J&M §5.4

Count vectors are meaningful but huge and sparse. Can we learn shorter, denser embeddings?

After training, SGNS becomes a lookup table

The training classifier can be discarded; the learned embedding matrices are the reusable result.

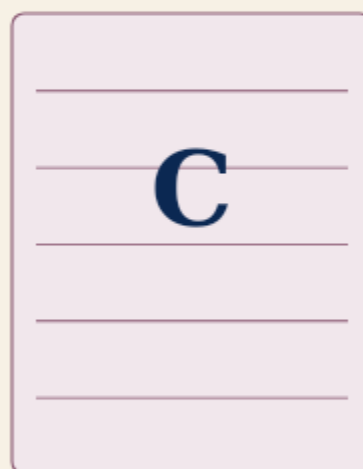
During training

Every vocabulary item i has two learned rows. Both matrices are updated by SGNS.

$$W, C \in \mathbb{R}^{|V| \times d}$$



target vectors w_i



context vectors c_i



after training

Choose one reusable vector per word

Common choice: combine the two roles

$$e_i = w_i + c_i$$

Alternative: retain only the target matrix

$$e_i = w_i$$

Ordinary use is now just a lookup

$$x \mapsto e_x \in \mathbb{R}^d$$

The sigmoid, positive labels and sampled noise pairs define the training problem.

No labels, negative samples, sigmoid or gradient descent are required for this lookup.

What can we compute once each word has a dense vector?

SGNS vectors become features for later tasks

A word vector can be compared directly, pooled into a document vector, or passed to another model.

Example text

“The treatment was very effective.”

Look up one learned vector for every token.

Compare words directly

Cosine similarity supports nearest neighbours, clustering and analogy-style inspection.

$$\cos(e_a, e_b) = \frac{e_a^T e_b}{\|e_a\| \|e_b\|}$$

effective → useful, successful, beneficial

Build a document feature

Pool the word vectors, then pass the resulting document vector to a classifier.

Simple average

$$d = \frac{1}{n} \sum_{i=1}^n e_{x_i}$$

TF-IDF-weighted average

$$d = \frac{\sum_i \text{tfidf}(x_i, d) e_{x_i}}{\sum_i \text{tfidf}(x_i, d)}$$

d → sentiment / spam / topic classifier

Preserve the sequence

Feed the ordered vectors into a CNN, RNN, LSTM or Transformer.

$$x_1, \dots, x_n \mapsto e_{x_1}, \dots, e_{x_n}$$

embedding sequence
↓
neural sequence model

Embedding layer

Freeze the pretrained vectors—or fine-tune them for the downstream task.

But TF-IDF already gives us vectors—what is fundamentally different?

TF-IDF and SGNS represent different objects

TF-IDF describes a document in vocabulary coordinates; SGNS describes a word in learned latent coordinates.

TF-IDF

$$v_d \in \mathbb{R}^{|V|}$$

Represents	a particular document or query d
Coordinates	named vocabulary terms
Values mean	importance of each term in d
Structure	high-dimensional and sparse
Across texts	the vector changes with the document
Direct use	document retrieval and document features

SGNS embedding

$$e_x \in \mathbb{R}^d$$

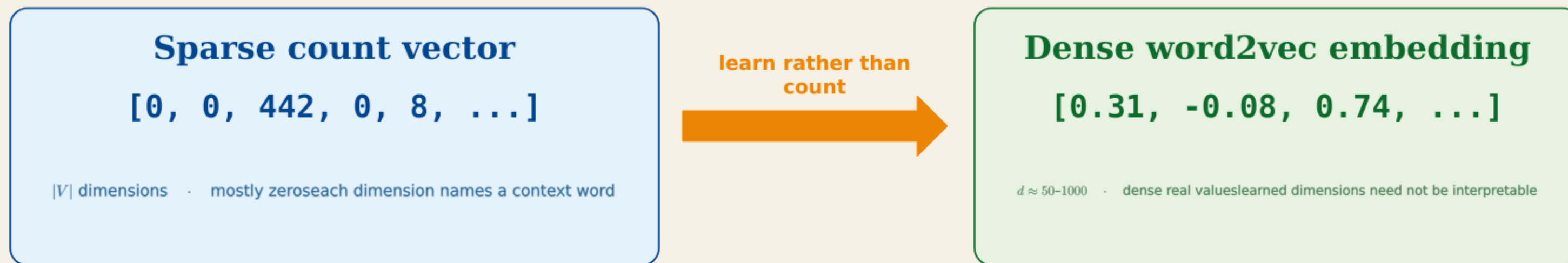
Represents	a vocabulary word type x
Coordinates	learned latent dimensions
Values mean	distributed properties learned from contexts
Structure	lower-dimensional and dense
Across texts	the same static vector is reused for x
Direct use	word similarity and token-level inputs

To represent a document with SGNS, we must combine its word vectors—for example by averaging or using a sequence model.

SGNS supplies word vectors; TF-IDF directly supplies a document or query vector.

Word2vec learns dense embeddings from context

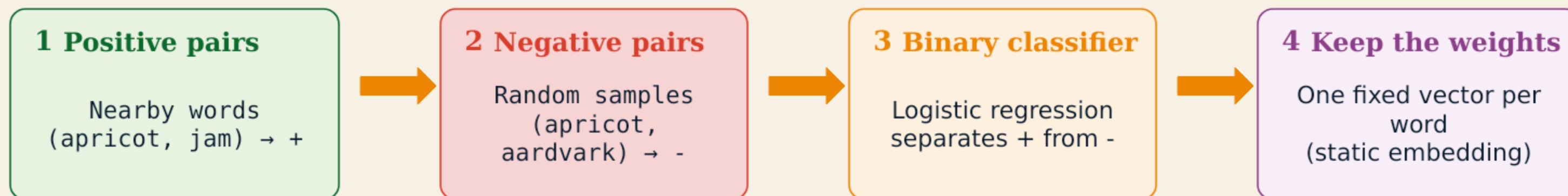
Running text supplies the labels; the learned classifier weights become word vectors.



Compact vectors reduce parameters and may capture **synonymy** better.

Skip-gram with negative sampling (SGNS)

Running text creates the labels:
self-supervision



J&M §5.5

How does the classifier compute $P(+ | w, c)$ from two embeddings?

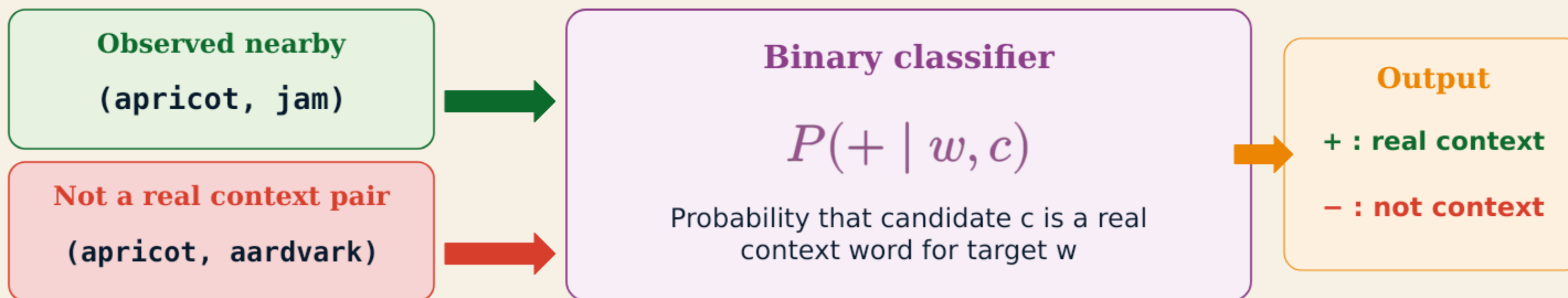
Skip-gram asks whether two words belong together

A target word and a candidate context word define one binary classification example.

A ± 2 context window supplies observed pairs



The classifier answers the same question for any candidate pair

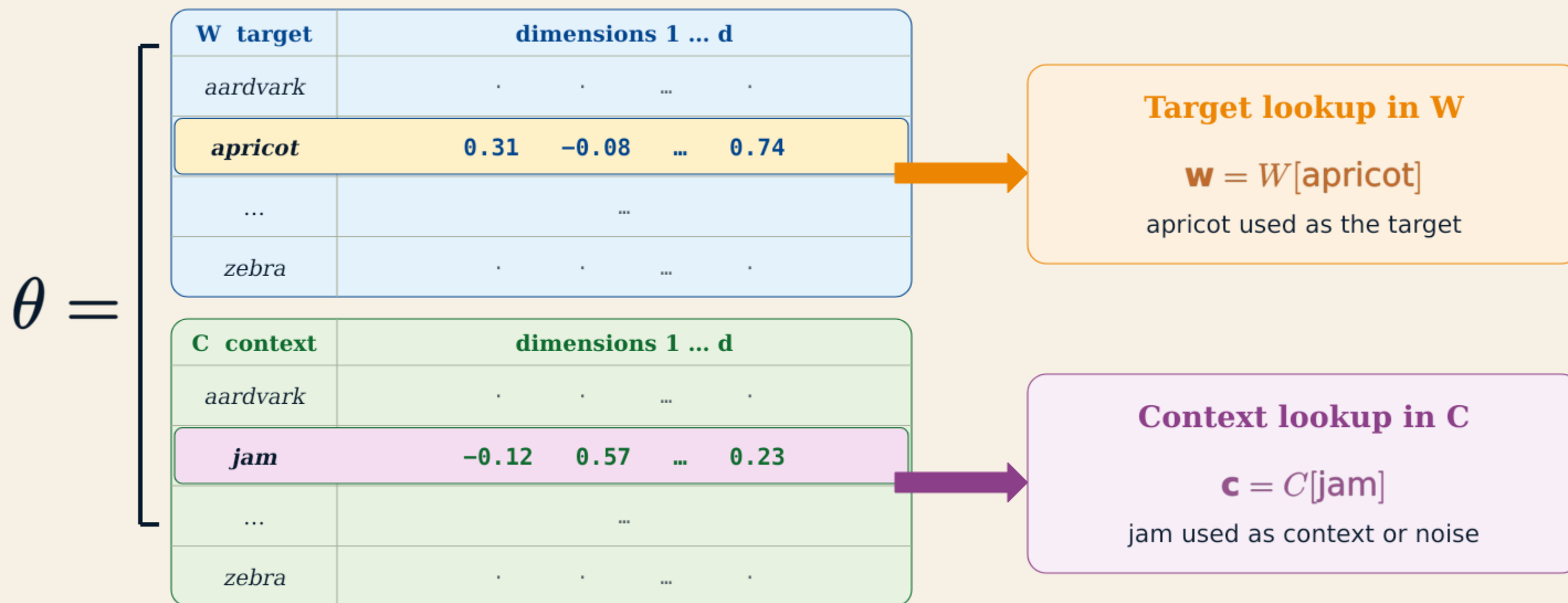


J&M §5.5.1

Where do the vectors w and c used by this classifier come from?

Every word has a target and a context embedding

Skip-gram learns separate parameter matrices W and C for these two roles.



Together: $2|V|$ learned vectors, each with d dimensions.

J&M Fig. 5.6 · redrawn

How can $\mathbf{w}$ and $\mathbf{c}$ express whether a target-context pair belongs together?

Dot products score target-context compatibility

A larger score supports the hypothesis that the pair occurs together in real text.

$$\text{score}(\mathbf{w}, \mathbf{c}) = \mathbf{c} \cdot \mathbf{w} = \sum_j c_j w_j$$

Multiply corresponding dimensions, then add the contributions.

Illustrative 3-D values

(apricot, jam)

$\mathbf{w} = [0.60, 0.20, 0.70]$

$\mathbf{c} = [0.50, 0.30, 0.60]$

$0.30 + 0.06 + 0.42$

$\mathbf{c} \cdot \mathbf{w} = 0.78$

aligned contributions → higher compatibility

(apricot, aardvark)

$\mathbf{w} = [0.60, 0.20, 0.70]$

$\mathbf{c} = [-0.40, 0.50, -0.30]$

$-0.24 + 0.10 - 0.21$

$\mathbf{c} \cdot \mathbf{w} = -0.35$

opposing contributions → lower compatibility

Recall: cosine is a normalized dot product. Skip-gram uses the **raw dot product**, so both direction and vector magnitude affect the score.

J&M §5.5.1 · Eq. 5.13

The score ranges from $-\infty$ to ∞ . How can it become a probability?

The sigmoid converts compatibility into probability

The dot products from the preceding slide now become probabilities of positive pairs.

Compatibility score

$$s = \mathbf{c} \cdot \mathbf{w}$$



$$\sigma(s) = \frac{1}{1 + \exp(-s)}$$



$$P(+ | w, c) \\ = \sigma(\mathbf{c} \cdot \mathbf{w})$$

Carry the same numerical scores through the transformation

apricot with jam

$$s = 0.78$$

$$\sigma(s) = 0.686$$

decision midpoint

$$s = 0.00$$

$$\sigma(s) = 0.500$$

apricot with matrix

$$s = -0.35$$

$$\sigma(s) = 0.413$$

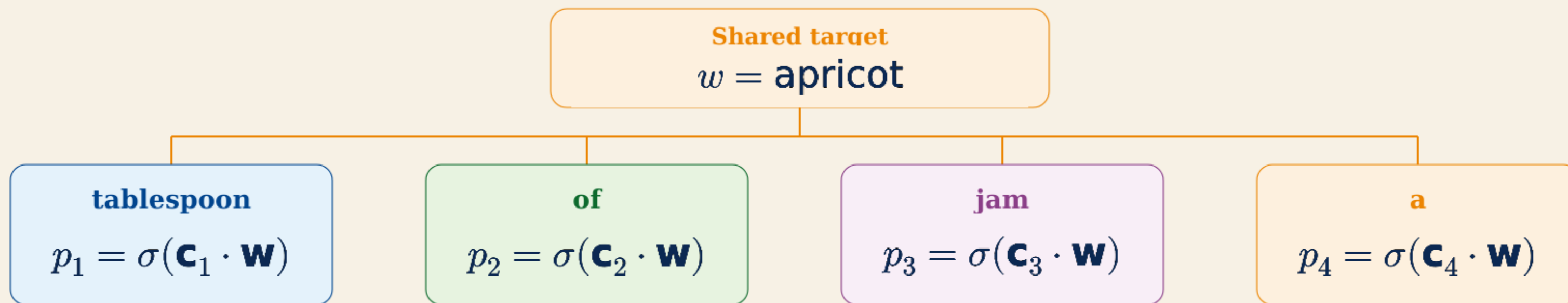
The sigmoid changes the scale, but preserves the ordering of the scores.

J&M §5.5.1 · Eqs. 5.14-5.16

One target has several context words. How are their probabilities combined?

A context window contributes several probabilities

Conditional independence lets skip-gram multiply the pair probabilities for one target.



Assumption: after the target is known, the context outcomes are treated as independent.

$$P(+ \mid w, c_{1:4}) = \prod_{i=1}^4 \sigma(\mathbf{c}_i \cdot \mathbf{w}) = p_1 p_2 p_3 p_4$$

J&M Eqs. 5.17-5.18

What transformation makes this product easier to optimize?

Logarithms turn the window product into a sum

The same objective becomes additive: every context word contributes one log-probability.

Window probability

$$P = p_1 p_2 p_3 p_4$$



take log

Log window probability

$$\log P = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

The transformation changes the calculation, not the preferred parameters

Multiplication becomes addition

Each context word contributes one separate term to the objective.

Gradients become additive

The target update can sum the contributions from all context words.

Because log is increasing, maximizing P also maximizes $\log P$.

J&M Eq. 5.18

How do these context pairs become positive and negative training examples?

A context window supplies positive and negative training pairs

Observed neighbours are positive; sampled vocabulary words provide contrasting negatives.

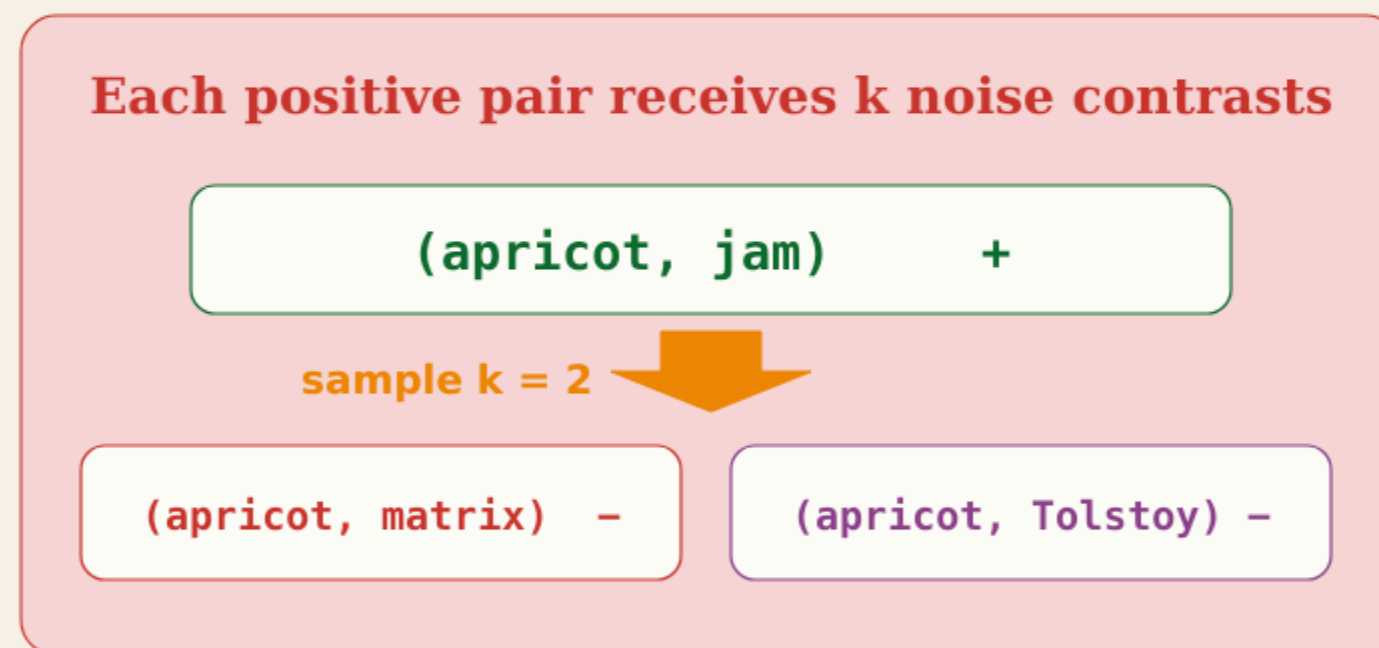
One ± 2 context window around the target



Observed neighbours $\rightarrow$ positive pairs



Each positive pair receives k noise contrasts



Four observed neighbours give four positive pairs and, with $k = 2$, eight noise pairs.

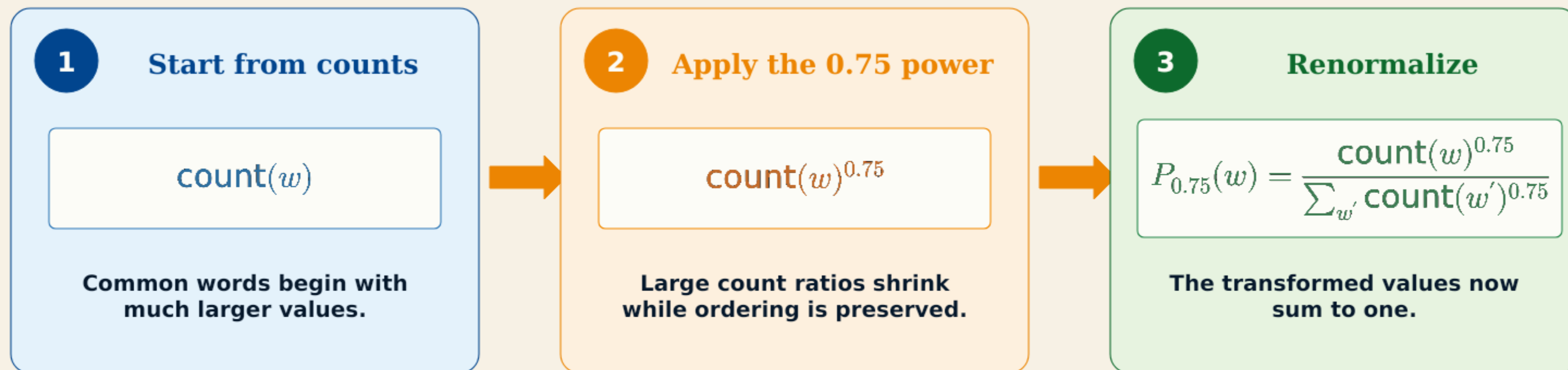
J&M §5.5.2

How should SGNS choose the noise words?

The 0.75 power flattens the noise distribution

SGNS transforms word counts and then renormalizes them into sampling probabilities.

Flatten first; normalize second



A 100:1 count ratio becomes $100^{0.75}:1^{0.75} \approx 31.6:1$ before normalization.

Frequent words remain more likely, but rare words receive more sampling mass.

J&M §5.5.2 · Eqs. 5.19-5.20

Once the positive and negative pairs are assembled, what objective trains W and C?

SGNS maximizes correct-label probability

One case contains one observed context word and k independently sampled noise words.

One sampled training case

target w

apricot

positive context c_{pos}

jam

noise context $c_{\text{neg},1}$

matrix

noise context $c_{\text{neg},2}$

Tolstoy

Observed pair should receive +

$$P(+ \mid w, c_{\text{pos}}) = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w})$$

Each noise pair should receive -

$$P(- \mid w, c_{\text{neg},i}) = \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

Correct-label likelihood

$$\mathcal{L} = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) \prod_{i=1}^k \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

J&M §5.5.2 · Eq. 5.21

How does taking negative logarithms turn this likelihood into a loss?

The negative label reverses the compatibility score

For a noise pair, the correct-class probability is the complement of the positive-class probability.

Start with the score for one sampled noise pair

1 Compatibility score

$$s_{\text{neg}} = c_{\text{neg}} \cdot w$$

2 Positive-class probability

$$P(+ \mid w, c_{\text{neg}}) = \sigma(s_{\text{neg}})$$

3 A negative label means “not positive”

$$P(- \mid w, c_{\text{neg}}) = 1 - P(+ \mid w, c_{\text{neg}}) = 1 - \sigma(s_{\text{neg}})$$

Sigmoid complement identity

$$1 - \sigma(s) = \sigma(-s)$$

Therefore

$$P(- \mid w, c_{\text{neg}}) = \sigma(-s_{\text{neg}}) = \sigma(-(c_{\text{neg}} \cdot w))$$

The outer minus negates the scalar score—not a separately stored context vector.

Numerical check

Suppose the noise pair already has a low compatibility score:

$$s_{\text{neg}} = -3$$

$$P(+) = \sigma(-3) = 0.047$$

$$P(-) = 1 - 0.047 = 0.953$$

$$P(-) = \sigma(3) = 0.953$$

A negative score gives a high probability to the negative label.

$$P(\text{correct}) = \sigma(y s)$$

$y = +1$ for observed; $y = -1$ for noise

How does taking the negative logarithm turn this likelihood into the SGNS loss?

The negative log gives the SGNS loss

The positive term and all k negative terms become additive contributions.

$$\mathcal{L} = \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) \prod_{i=1}^k \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w})$$

take $-\log$ 

SGNS loss

$$L = - \left[\log \sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) + \sum_{i=1}^k \log \sigma(-\mathbf{c}_{\text{neg},i} \cdot \mathbf{w}) \right]$$

Positive term

Minimizing L increases $\mathbf{c}_{\text{pos}} \cdot \mathbf{w}$.

Negative terms

Minimizing L decreases every $\mathbf{c}_{\text{neg},i} \cdot \mathbf{w}$.

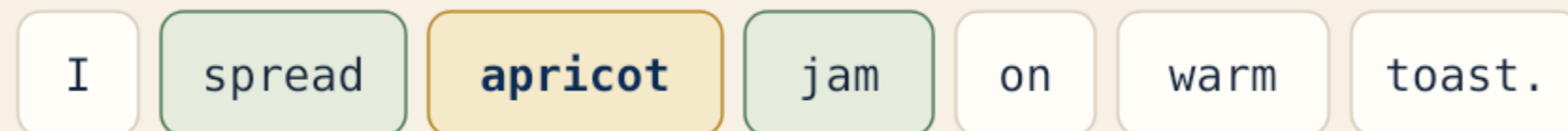
J&M §5.5.2 · Eq. 5.21

What does one complete forward pass look like with actual numbers?

Running text supplies the positive examples

A context window converts one sentence into labelled target-context pairs.

Sentence



window = 1 word on each side

observed near apricot

The target word is paired with every word inside its window

(apricot, spread)

$y = 1$

observed immediately before

(apricot, jam)

$y = 1$

observed immediately after

These labels come from the text itself: no human annotator is required.

J&M §5.5.2

A classifier also needs examples of words that were not observed nearby—where do they come from?

Negative sampling creates the contrasting examples

Keep one observed pair, then sample words that did not occupy that context position.

One small training batch

(apricot, jam)

$y = 1$

meaning of the label

jam occurred beside apricot

(apricot, aardvark)

$y = 0$

sampled noise word

(apricot, tractor)

$y = 0$

sampled noise word

(apricot, ocean)

$y = 0$

sampled noise word

“Negative” means not observed in this context window. It does not mean the words can never be related.

J&M §5.5.2

How are these word identities converted into quantities the classifier can score?

Each labelled pair selects two embedding rows

The target comes from W ; every candidate context word comes from C .

Target lookup

$W[\text{apricot}]$

$\mathbf{w} = [0.60, 0.20, 0.70]$

the same target row is reused

Context lookups

$C[\text{jam}]$

$[0.50, 0.30, 0.60]$

$C[\text{aardvark}]$

$[-0.40, 0.50, -0.30]$

$C[\text{tractor}]$

$[-0.50, -0.20, 0.10]$

$C[\text{ocean}]$

$[0.10, -0.50, -0.40]$

Four pairs $\rightarrow$ one W row and four C rows. The word apricot is not re-embedded four times.

J&M §5.5.2

How can two selected vectors yield a single compatibility score?

Dot products score the four candidate pairs

For each pair, multiply matching dimensions and add the three contributions.

$$\text{score}(\mathbf{w}, \mathbf{c}) = \mathbf{c} \cdot \mathbf{w}$$

jam

$$0.50(0.60) + 0.30(0.20) + 0.60(0.70) = 0.30 + 0.06 + 0.42$$

$$\text{score} = 0.78$$

aardvark

$$-0.40(0.60) + 0.50(0.20) - 0.30(0.70) = -0.24 + 0.10 - 0.21$$

$$\text{score} = -0.35$$

tractor

$$-0.50(0.60) - 0.20(0.20) + 0.10(0.70) = -0.30 - 0.04 + 0.07$$

$$\text{score} = -0.27$$

ocean

$$0.10(0.60) - 0.50(0.20) - 0.40(0.70) = 0.06 - 0.10 - 0.28$$

$$\text{score} = -0.32$$

J&M §5.5.2

The scores range from $-\infty$ to ∞ . How do they become probabilities?

The sigmoid turns each score into a pair probability

Apply the same nonlinear transformation independently to all four scores.

$$p(y = 1 \mid w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) \quad \text{where} \quad \sigma(s) = \frac{1}{1 + \exp(-s)}$$

pair	score s	$\sigma(s)$	label y	reading
(apricot, jam)	0.78	0.686	1	<i>higher is better</i>
(apricot, aardvark)	-0.35	0.413	0	<i>lower is better</i>
(apricot, tractor)	-0.27	0.433	0	<i>lower is better</i>
(apricot, ocean)	-0.32	0.421	0	<i>lower is better</i>

The sigmoid changes the scale but preserves the ordering of the scores.

J&M §5.5.2

How are the four prediction errors combined into one training objective?

The forward pass ends with one combined SGNS loss

Reward the observed pair for high probability and every noise pair for low probability.

Observed pair

$$-\log \sigma(\mathbf{c}_{\text{jam}} \cdot \mathbf{w})$$

$$= -\log(0.685680)$$

$$= 0.377344$$

small when the positive probability is high

Three noise pairs

$$\text{aardvark} \quad -\log(1 - \sigma(-0.35)) = 0.533382$$

$$\text{tractor} \quad -\log(1 - \sigma(-0.27)) = 0.567232$$

$$\text{ocean} \quad -\log(1 - \sigma(-0.32)) = 0.545893$$

$$\text{noise subtotal} = 1.646507$$

$$\mathbf{L} = 0.377344 + 1.646507 = 2.023851 \approx 2.024$$

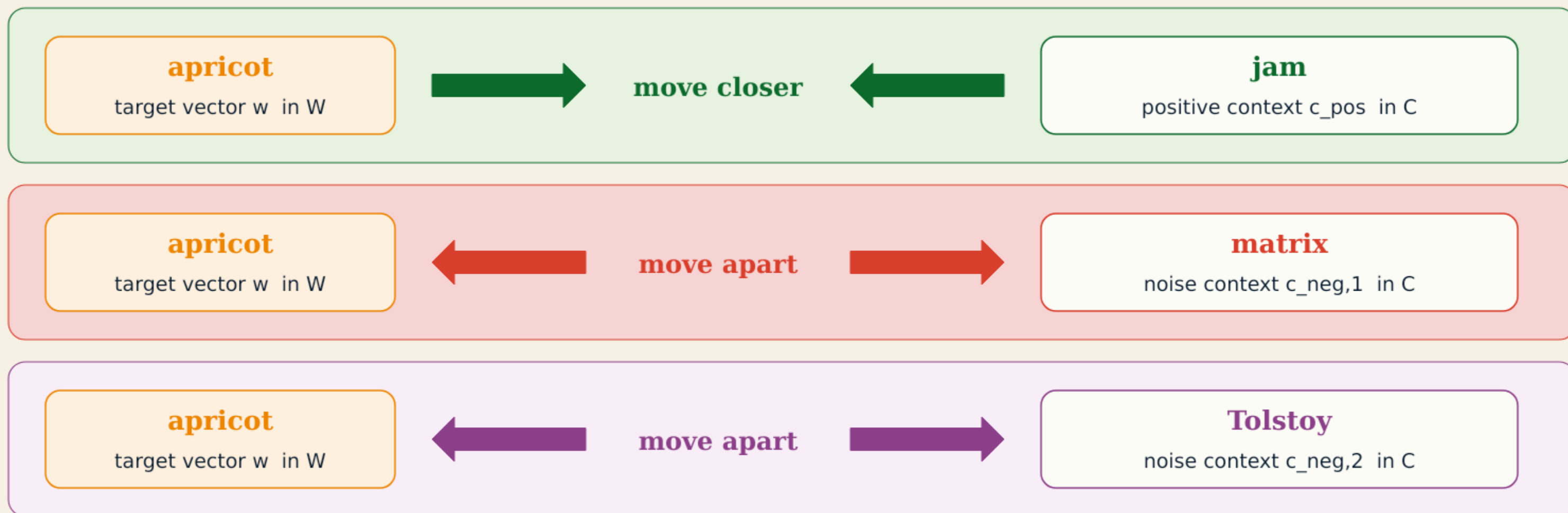
Forward pass complete: no parameter has been changed yet.

J&M §5.5.2 • Eq. 5.20

Which vectors should move—and in what direction—to reduce this loss?

SGD pulls neighbours closer and repels noise

Figure 5.7 visualizes how one update changes the sampled target and context vectors.



One target update combines the **positive pull** and both **negative pushes**.

J&M Fig. 5.7 · redrawn

The geometry is intuitive. What scalar coefficients determine each movement size?

Prediction error controls the size of each gradient

The partner embedding sets the direction; the sigmoid coefficient sets the magnitude.

gradient = scalar prediction error $\times$ partner embedding

$$\frac{\partial L}{\partial \mathbf{c}_{\text{pos}}}$$

$$[\sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) - 1] \mathbf{w}$$

negative coefficient
→ pull toward $\mathbf{w}$

$$\frac{\partial L}{\partial \mathbf{c}_{\text{neg},i}}$$

$$\sigma(\mathbf{c}_{\text{neg},i} \cdot \mathbf{w}) \mathbf{w}$$

positive coefficient
→ push away from $\mathbf{w}$

$$\frac{\partial L}{\partial \mathbf{w}}$$

$$[\sigma(\mathbf{c}_{\text{pos}} \cdot \mathbf{w}) - 1] \mathbf{c}_{\text{pos}} + \sum_i \sigma(\mathbf{c}_{\text{neg},i} \cdot \mathbf{w}) \mathbf{c}_{\text{neg},i}$$

combined pull
and k pushes

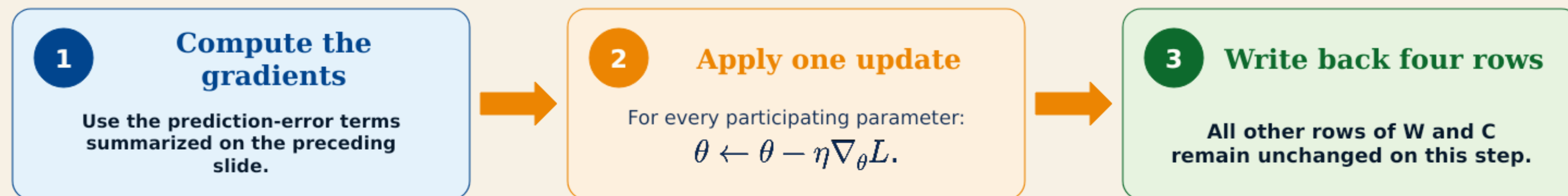
Positive pair: $p_{\text{pos}} - 1 \in (-1, 0)$ Negative pair: $p_{\text{neg}} \in (0, 1)$

J&M Eqs. 5.22-5.24

How are these gradients applied to $\mathbf{W}$ and $\mathbf{C}$ during one corpus pass?

Each SGD step changes only the sampled rows

The standard gradient-descent rule is applied locally to the target and sampled contexts.



Generic SGD rule

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

Rows touched by the apricot training case

$W[\text{apricot}]$

$C[\text{jam}]$

$C[\text{matrix}]$

$C[\text{Tolstoy}]$

Sparse row updates make training efficient even with a large vocabulary.

J&M Eqs. 5.25-5.27

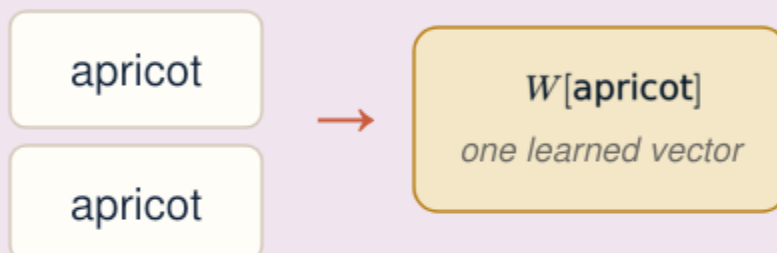
Once the vectors are learned, what determines the kind of similarity they encode?

SGNS begins by deciding what counts as evidence

The corpus supplies labels only after we define the represented unit, neighborhood and training pairs.

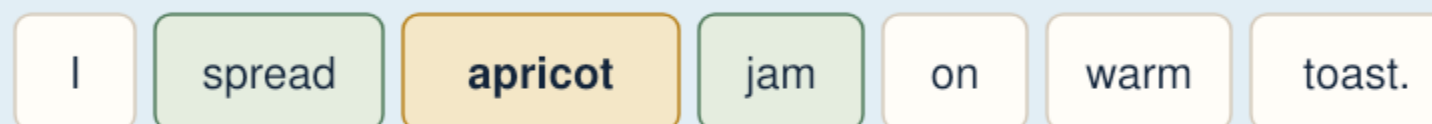
1 · Represent word types

Different occurrences share one static representation.



2 · Define the neighborhood

The context-window radius is a modeling choice.



In basic SGNS, position and order are discarded after the pairs are formed.

window = 1 word on each side

3 · Turn proximity into self-supervised labels

Observed nearby

(apricot, spread) $y = 1$
(apricot, jam) $y = 1$

One window → several pairs

Each context position creates its own positive training instance.

Each pair is judged separately

The model will make an independent binary decision for every target–context pair.

J&M §5.5–5.5.2

Once positive evidence has been defined, where do the contrasting examples come from?

Negative sampling defines the contrastive problem

Deliberately constructed non-neighbors complement the positive pairs supplied by text.

4 · Choose the prediction task

Binary pair classification

“Did this context occur near this target?”

SGNS does not predict a normalized distribution over the full vocabulary.

5–6 · Manufacture k negative pairs

one observed pair

(apricot, jam) $y = 1$

More negatives strengthen the contrast but increase computation.

k = 3 sampled noise pairs

(apricot, aardvark) $y = 0$

(apricot, tractor) $y = 0$

(apricot, ocean) $y = 0$

7–8 · Choose the noise distribution and preserve pairwise independence

$$P_{\alpha}(c) = \frac{\text{count}(c)^{\alpha}}{\sum_{c'} \text{count}(c')^{\alpha}}$$

Typical choice: $\alpha = 0.75$

Flattening the unigram distribution gives rare words relatively more sampling probability.

Every positive or negative pair receives its own sigmoid and contributes its own term to the loss.

Negative sampling is part of the learning problem—not merely a computational shortcut.

J&M §5.5–5.5.2

What parameterization makes these independent pairwise decisions learnable?

SGNS turns contrastive decisions into embeddings

The remaining choices determine the score, objective, sparse update and representation used downstream.

9–10 · Choose capacity and store two roles for every word

$$\begin{array}{ccc} W[w] & + & C[w] \\ \text{target vector} & & \text{context vector} \end{array}$$

Two $|V| \times d$ parameter tables

Embedding dimension d controls capacity and cost.

static: one W row and one C row per word type

11–13 · Choose the score, probability and objective

$$s = C[c] \cdot W[w]$$

dot-product compatibility



$$p = \sigma(s)$$

independent binary decision



binary cross-entropy

one positive + k negatives

14 · Update only selected rows

$$W[w] \cdot C[c^+] \cdot \text{sampled } C[c^-]$$

All other rows remain untouched in this training example.

15 · Choose the exported word vector

$$W[w] \quad C[w] \quad W[w] + C[w]$$

The final representation is a downstream modeling decision.

Training configuration: random initialization · learning rate · epochs · batching · corpus order · vocabulary cutoff · frequent-word subsampling J&M §5.5–5.5.2

SGNS is a designed contrastive learning system: its embeddings inherit every choice made above.

Context window size changes what similar means

Nearby evidence emphasizes functional similarity; wider evidence emphasizes topic.

The context window is a semantic design choice

Short window: ± 2

same-kind / often same part of speech

Hogwarts

Sunnydale · Evernight

Other fictional schools
functionally similar to Hogwarts

Longer window: ± 5

broader topical association

Hogwarts

**Dumbledore · Malfoy
half-blood**

Words from the wider Harry Potter topic

Changing the window changes the **relation encoded** by the embedding.

J&M §5.7 · Levy & Goldberg (2014a)

Are words related because they occur together, or because they occur in similar contexts?

Co-occurrence encodes two kinds of association

Direct neighbours and shared neighbourhoods encode different semantic evidence.

The difference is whether words share a sentence neighbourhood or share the same neighbours

**First-order
co-occurrence**
syntagmatic association

book

near
each other

wrote

poem

The words typically occur in the same local context.

**Second-order
co-occurrence**
paradigmatic association

said

similar
neighbours

wrote

remarked

The words need not occur together; their context distributions resemble one another.

together versus **in similar company**

J&M §5.7 · Schütze & Pedersen (1993)

Can a repeated relation appear as a consistent direction in embedding space?

Vector offsets can encode relations

A shared displacement links corresponding members of an analogy.

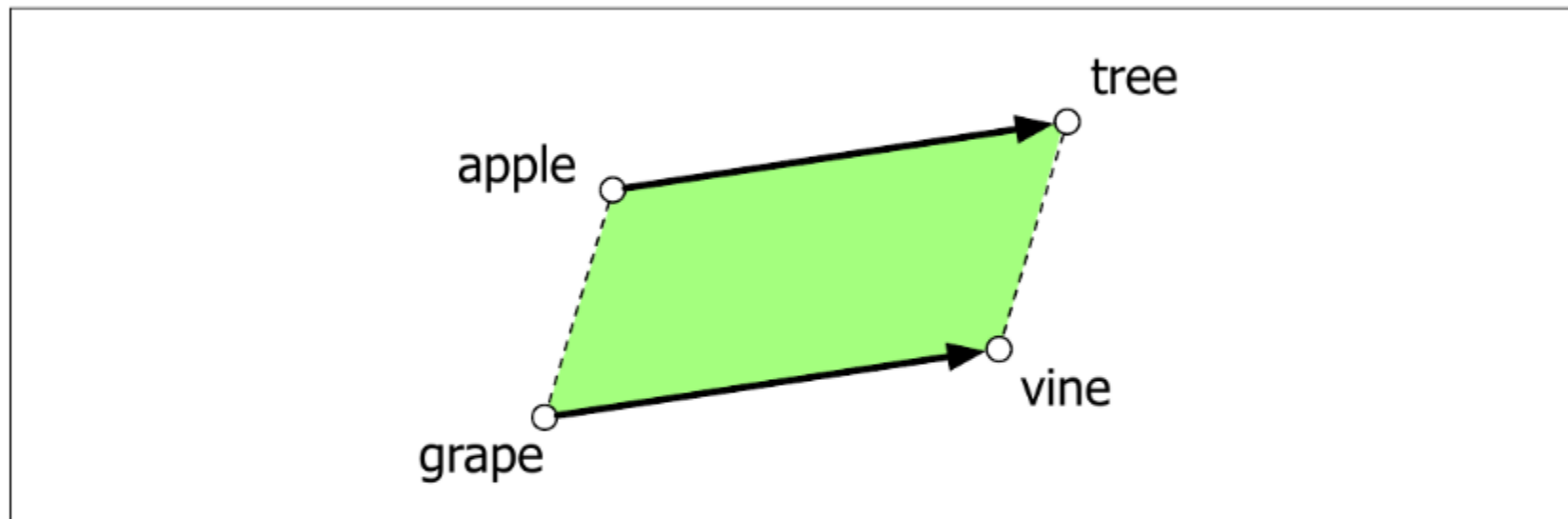


Figure 5.8 The parallelogram model for analogy problems (Rumelhart and Abrahamson, 1973): the location of $\vec{vine}$ can be found by subtracting $\vec{apple}$ from $\vec{tree}$ and adding $\vec{grape}$.

The relation is a displacement

$$\vec{tree} - \vec{apple} \approx \vec{vine} - \vec{grape}$$

$$\vec{tree} - \vec{apple} + \vec{grape} \approx \vec{vine}$$

Nearest-neighbour search returns the word closest to the predicted point.

J&M Fig. 5.8 · original

When does the parallelogram model succeed - and where does it fail?

Analogy structure is real, but not universal

Offsets work best for frequent words, short distances, and selected relations.

What the result requires

Control the candidates

Exclude the three input words and their morphological variants.

Expect selective success

Frequent words, country-capital pairs, and inflectional relations work best.

Do not overinterpret

Many relations do not form clean parallelograms; human analogy is richer.

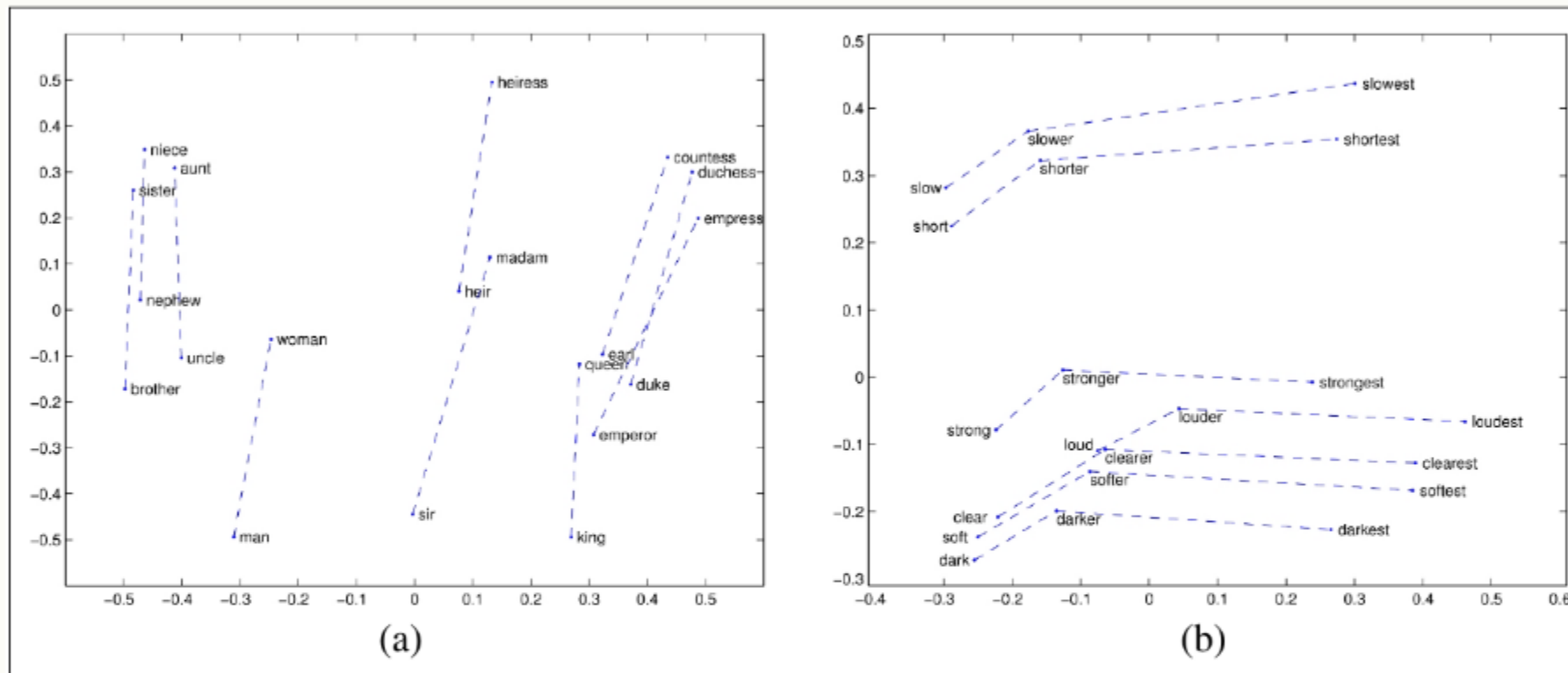


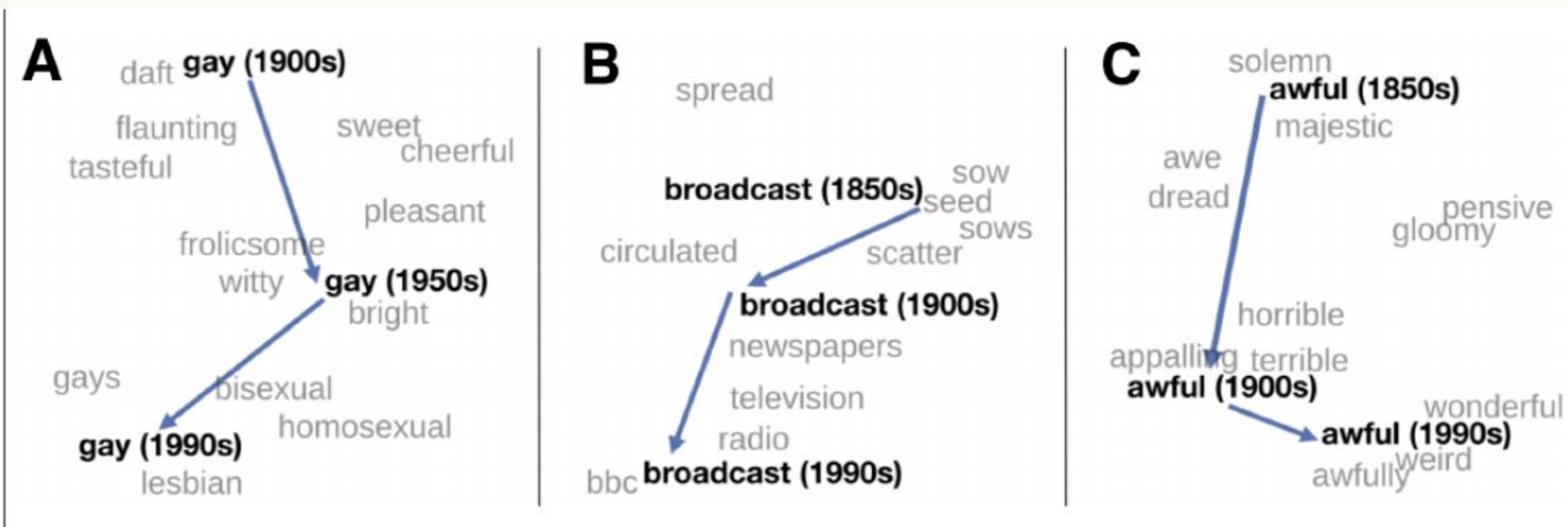
Figure 5.9 Relational properties of the GloVe vector space, shown by projecting vectors onto two dimensions. (a) $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}}$ is close to $\vec{\text{queen}}$. (b) offsets seem to capture comparative and superlative morphology (Pennington et al., 2014).

J&M Fig. 5.9 and §5.7 · original

How can embedding geometry reveal that a word's meaning has changed over time?

Embedding spaces reveal semantic change

Train one space per historical period, then compare a word's neighbourhood over time.



gay

cheerful / frolicsome
→ homosexuality

broadcast

sowing seeds
→ transmitting signals

awful

full of awe
→ terrible / appalling

J&M Fig. 5.10 · Hamilton et al. (2016b)

After training, SGNS becomes a lookup table

The training classifier can be discarded; the learned embedding matrices are the reusable result.

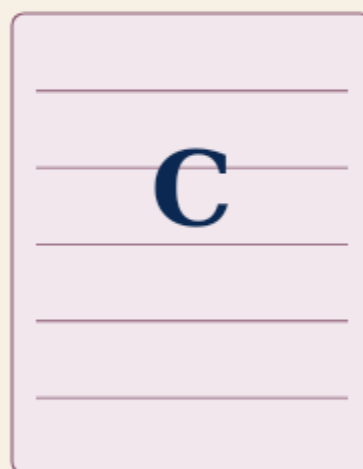
During training

Every vocabulary item i has two learned rows. Both matrices are updated by SGNS.

$$W, C \in \mathbb{R}^{|V| \times d}$$



target vectors w_i



context vectors c_i



after training

The sigmoid, positive labels and sampled noise pairs define the training problem.

Choose one reusable vector per word

Common choice: combine the two roles

$$e_i = w_i + c_i$$

Alternative: retain only the target matrix

$$e_i = w_i$$

Ordinary use is now just a lookup

$$x \mapsto e_x \in \mathbb{R}^d$$

No labels, negative samples, sigmoid or gradient descent are required for this lookup.

What can we compute once each word has a dense vector?

SGNS vectors become features for later tasks

A word vector can be compared directly, pooled into a document vector, or passed to another model.

Example text

“The treatment was very effective.”

Look up one learned vector for every token.

Compare words directly

Cosine similarity supports nearest neighbours, clustering and analogy-style inspection.

$$\cos(e_a, e_b) = \frac{e_a^T e_b}{\|e_a\| \|e_b\|}$$

effective → useful, successful, beneficial

Build a document feature

Pool the word vectors, then pass the resulting document vector to a classifier.

Simple average

$$d = \frac{1}{n} \sum_{i=1}^n e_{x_i}$$

TF-IDF-weighted average

$$d = \frac{\sum_i \text{tfidf}(x_i, d) e_{x_i}}{\sum_i \text{tfidf}(x_i, d)}$$

d → sentiment / spam / topic classifier

Preserve the sequence

Feed the ordered vectors into a CNN, RNN, LSTM or Transformer.

$$x_1, \dots, x_n \mapsto e_{x_1}, \dots, e_{x_n}$$

embedding sequence
↓
neural sequence model

Embedding layer

Freeze the pretrained vectors—or fine-tune them for the downstream task.

But TF-IDF already gives us vectors—what is fundamentally different?

TF-IDF and SGNS represent different objects

TF-IDF describes a document in vocabulary coordinates; SGNS describes a word in learned latent coordinates.

TF-IDF

$$v_d \in \mathbb{R}^{|V|}$$

Represents	a particular document or query d
Coordinates	named vocabulary terms
Values mean	importance of each term in d
Structure	high-dimensional and sparse
Across texts	the vector changes with the document
Direct use	document retrieval and document features

SGNS embedding

$$e_x \in \mathbb{R}^d$$

Represents	a vocabulary word type x
Coordinates	learned latent dimensions
Values mean	distributed properties learned from contexts
Structure	lower-dimensional and dense
Across texts	the same static vector is reused for x
Direct use	word similarity and token-level inputs

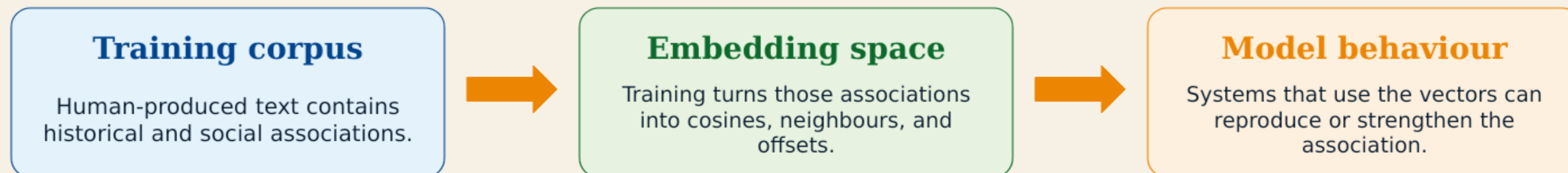
To represent a document with SGNS, we must combine its word vectors—for example by averaging or using a sequence model.

SGNS supplies word vectors; TF-IDF directly supplies a document or query vector.

Embeddings can amplify social bias

Associations in human-produced text become geometric relations in the learned space.

Corpus associations become embedding geometry



Stereotyped relations reported in word2vec embeddings

computer programmer – man + woman $\approx$ homemaker

father : doctor :: mother : nurse

These are learned corpus associations - not valid prescriptions about people.

J&M §5.8 · Bolukbasi et al. (2016)

When do learned associations become real-world harms?

Embedding bias can produce two kinds of harm

Allocational harm affects opportunities; representational harm affects how groups are portrayed.

Embedding similarity influences ranking, retrieval, and classification

Allocational harm

Resources or opportunities are distributed unfairly across groups.

Example: hiring search

Embedding-based ranking may downweight documents associated with women when searching for programmers or doctors.

Representational harm

A group is stereotyped, demeaned, excluded, or made less visible.

Example: sentiment association

Cosine similarity between names and pleasant or unpleasant words can reproduce implicit social associations.

The harm depends on how the embedding is used - not only on the vector itself.

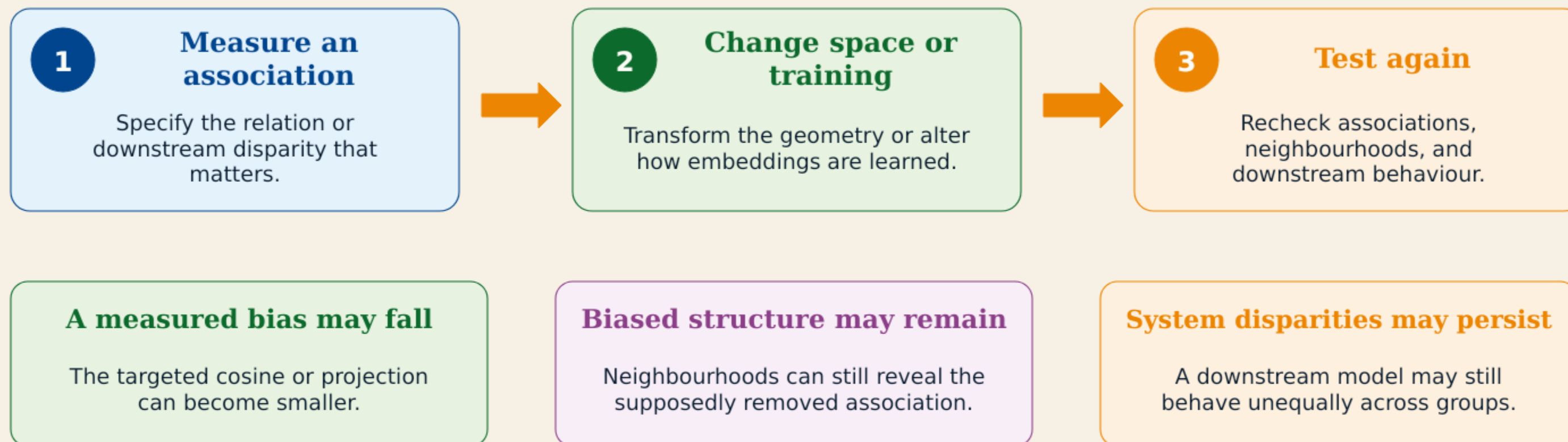
J&M §5.8 · Crawford (2017) · Blodgett et al. (2020)

Can we remove unwanted associations while preserving useful semantic structure?

Debiasing does not eliminate embedding bias

A transformed embedding can score better on one bias test while retaining biased structure elsewhere.

Debiasing is an intervention followed by evaluation



Debiasing is something to evaluate - not a certificate of fairness.

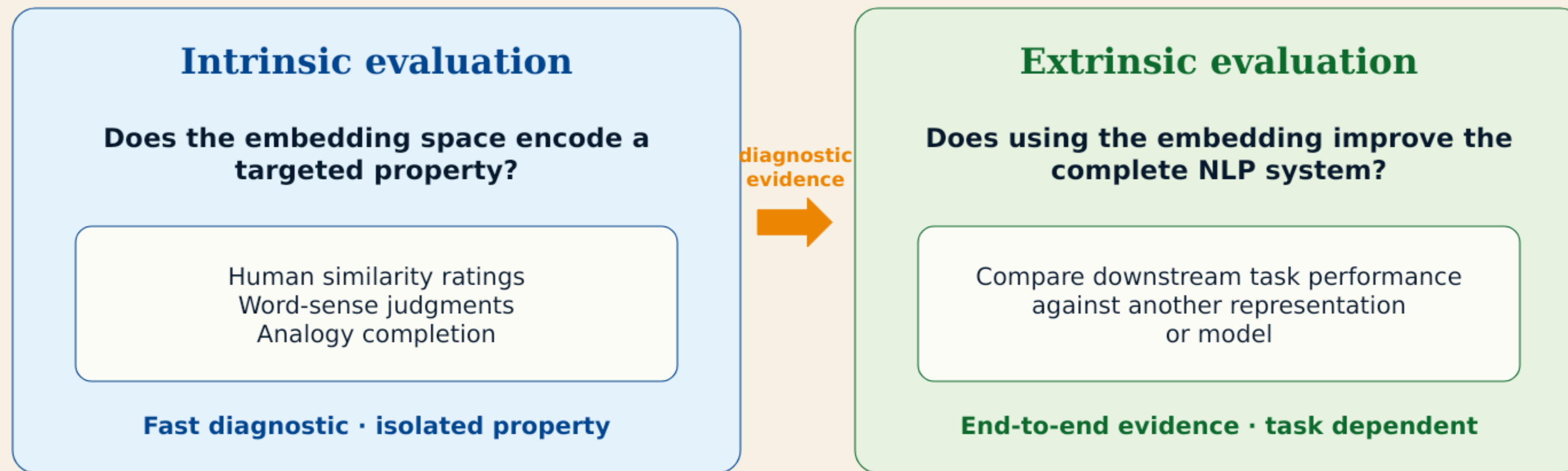
J&M §5.8 · Gonen & Goldberg,
2019

Should we evaluate the vector space, the downstream task, or both?

Extrinsic evaluation is the decisive test

Intrinsic tests inspect vector geometry; extrinsic tests ask whether embeddings improve an NLP task.

Two levels of evidence answer different questions



Intrinsic and extrinsic scores need not agree.

J&M §5.9

If intrinsic evaluation is a diagnostic, which semantic properties should it test?

Intrinsic benchmarks test distinct properties

A high score is meaningful only when the benchmark matches the property we care about.

Input being judged

Representative datasets

Property tested

Words without context

WordSim-353 · SimLex-999 · TOEFL

Relatedness, similarity, or synonym choice

Meaning in context

**SCWS · WiC · semantic textual
similarity**

Contextual similarity, word sense, or
sentence similarity

Relational structure

Analogy datasets

Morphological, lexical, and encyclopedic
relations

J&M §5.9

Would the same embedding produce the same result if we trained it again?

Embedding evaluations must account for variability

Randomness and corpus composition can change the learned neighbourhoods and measured associations.

Different training runs can produce different embedding spaces

Random initialization

Different starting points alter the optimization trajectory.

Negative sampling

Different noise words change the updates seen during training.

Corpus composition

Individual documents can strongly affect learned associations.

Robust practice checks whether a pattern is reproducible

1 Train multiple spaces

Repeat training with different random seeds.

2 Bootstrap documents

Resample documents to test sensitivity to the corpus.

3 Summarize variability

Average results and report how much they vary across runs.

Report variability across runs, not only the average.

J&M §5.9 · Antoniak & Mimno (2018)

Trust semantic claims that remain stable across document samples, random seeds, and downstream tasks.